OBJECT-ORIENTED MODELING (OOM) PROJECT REPORT

Passbook

Bank Management System

A web application in plain Java 17 with SQLite, designed and documented with UML

Submitted by	Palaparthi Santhosh
Roll number	IIT2025160
Course	Object-Oriented Modeling (OOM)
Project	Passbook: Bank Management System (FR1-FR19)
Technology	Java 17 (no framework), SQLite via JDBC, HTML + CSS front end
Report date	November 2026

This report describes the design of the Passbook project, presents its UML diagrams (use case, class, component, sequence, collaboration, state, object and database diagrams), and explains where each object-oriented concept (abstraction, encapsulation, inheritance, polymorphism, abstract classes and interfaces) is used in the source code.

Table of Contents

1. Introduction 2

1.1 Project at a glance . . . 2

1.2 Demo logins . . . 2

2. Requirements and Where They Are Implemented 2

3. System Architecture 4

3.1 Package map . . . 5

3.2 How one request travels . . . 5

4. UML Diagrams 6

4.1 Use case diagram . . . 6

4.2 Class diagrams . . . 7

4.3 Sequence diagrams . . . 10

4.4 State diagrams . . . 12

4.5 Database design (ER diagram) . . . 13

4.6 Component diagram . . . 14

4.7 Collaboration diagram . . . 15

4.8 Object diagram . . . 16

5. Object-Oriented Concepts and Where They Are Used 17

5.1 Abstraction . . . 17

5.2 Encapsulation . . . 17

5.3 Inheritance . . . 18

5.4 Polymorphism . . . 19

5.5 Abstract Classes . . . 20

5.6 Interfaces . . . 20

5.7 Other OOP and Java Features Used . . . 21

6. Exception Handling 21

7. Design Patterns Used 22

8. Security and Reliability Features 22

9. Testing 23

10. Limitations and Future Scope 23

11. Conclusion 24

Appendix A. Source File Inventory 24

1. Introduction

Passbook is a bank management system. A bank has customers who hold savings and current accounts, move money between them, borrow through loans, and are served by tellers and managers. Passbook models these things as Java objects, stores them in a SQLite database, and exposes them as a web application that runs on the user's own computer at **http://127.0.0.1:8080**.

The project has two goals. First, it is a working application that implements the requirements FR1 to FR19 from the project guide: account opening, deposits, withdrawals, atomic transfers, statements, loans with EMI calculation, monthly interest, role-based access, login lockout, staff administration and an audit trail. Second, it is an exercise in object-oriented modeling: the code is organised so that every core OOP concept appears in a place where it solves a real problem, not as decoration.

1.1 Project at a glance

Item	Detail
Language and runtime	Java 17, plain JDK (built-in `com.sun.net.httpserver` web server, no Spring or other framework)
Database	SQLite through JDBC; file `bank.db` created on first start with demo data
Front end	Server-side generated HTML and a hand-written stylesheet (`static/style.css`)
Size	About 4,700 lines of Java across 6 main packages (model, exception, repo, service, security, web) plus util and App
Roles	Customer, Teller, Manager (Manager is a Teller with extra powers)
Tests	31 automated tests (TC1-TC9 from the guide, loans, interest, rollback, concurrency, and the web layer over real HTTP)
Build and run	`mvn -q compile exec:java`, or `./run.sh` (Windows: `run.bat`)

1.2 Demo logins

Role	User ID	Password
Customer	alice	alice123
Teller	teller1	teller123
Manager	admin	admin123

The demo data contains one manager, one teller, and one customer (Alice Verma) who owns a savings account and a current account.

2. Requirements and Where They Are Implemented

Every functional requirement is implemented by a specific service method. The table below maps requirements to the code, so a reader can go from a requirement to the class that fulfils it.

FR	Requirement	Implemented in
FR1-FR2	Open an account (new or existing customer), with opening deposit rules	AccountService.openAccount(), AccountFactory
FR3	Search and update customer profiles	AccountService.searchCustomers(), updateProfile(); CustomerProfile record validates itself

FR	Requirement	Implemented in
FR4	Close an account (balance must be zero, no active loan)	AccountService.close(), Account.close()
FR5-FR6	Deposit and withdraw, with account-type rules	Account.deposit(), Account.withdraw(); SavingsAccount / CurrentAccount.canWithdraw()
FR7	Atomic transfer between accounts	AccountService.transfer() inside UnitOfWork
FR8	Every balance change has a ledger entry	AccountService.post() saves balance and Transaction together
FR10-FR11	Mini-statement (last 10) and statement by date range	AccountService.miniStatement(), statement()
FR12-FR13	Apply for a loan; EMI is calculated	LoanService.apply(), Loan.apply(), Loan.calculateEmi()
FR14	Manager approves or rejects a loan	LoanService.decide(), Loan.approve() / reject()
FR15	Repay a loan	LoanService.repay(), Loan.repay()
FR16	Credit monthly interest to savings accounts	AccountService.creditMonthlyInterest(), InterestBearing
FR17	Login with 3-strike lockout	AuthService.login(), User.registerFailedAttempt()
FR18	Manager creates staff, disables users, unlocks users	AdminService.createEmployee(), setActive(), unlock()
FR19	Audit trail and branch statistics	EventBus + AuditListener; AdminService.auditTrail(), stats()

Non-functional requirements covered: salted password hashing (PBKDF2), CSRF protection on every POST, HTML escaping against XSS, session timeout after 10 idle minutes, role-based authorization on every route, and consistent balances under concurrent requests.

3. System Architecture

The program is built in layers. A request enters at the top, passes through checks, is handled by a controller, which calls a service. The service applies business rules using domain objects and saves them through repository interfaces. Each layer only knows the layer directly below it, and the layer below never knows about the layer above.

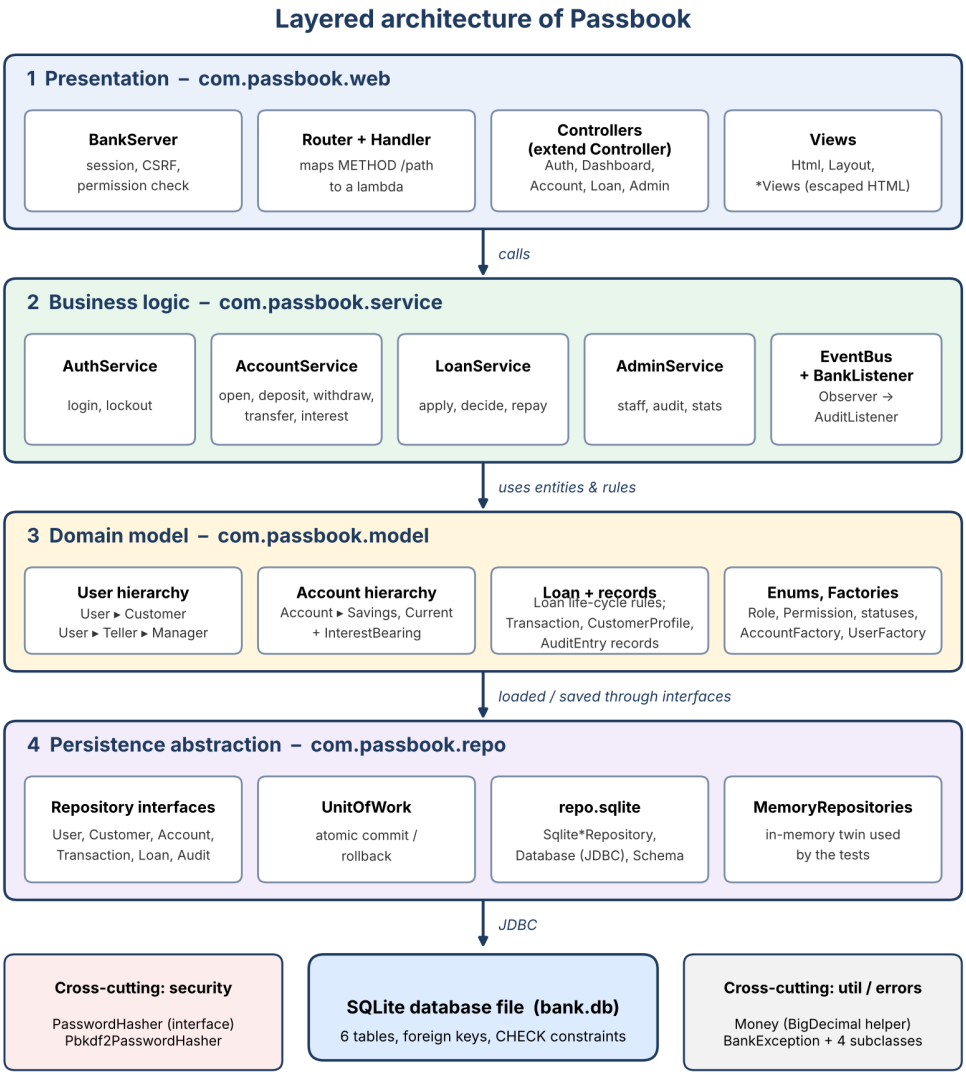


Figure 3.1: Layered architecture and package responsibilities

3.1 Package map

Package	Responsibility	Main classes
com.passbook.model	Domain objects and their rules	User, Customer, Teller, Manager, Account, SavingsAccount, CurrentAccount, Loan, Transaction, CustomerProfile, AuditEntry, enums, factories
com.passbook.exception	Business errors	BankException and four subclasses
com.passbook.repo	Storage interfaces	UserRepository, AccountRepository, CustomerRepository, TransactionRepository, LoanRepository, AuditRepository, UnitOfWork, Repositories
com.passbook.repo.sqlite	SQLite implementation	Database, Schema, Sqlite*Repository, SqliteUnitOfWork, SqliteRepositories
com.passbook.service	Business logic	AuthService, AccountService, LoanService, AdminService, EventBus, BankListener, AuditListener, DemoData
com.passbook.security	Password hashing	PasswordHasher (interface), Pbkdf2PasswordHasher
com.passbook.web	HTTP, routing, pages	BankServer, Router, Handler, Controller and 5 controllers, Views, Session, SessionStore
com.passbook	Wiring and start-up	App (composition root, main method)

3.2 How one request travels

A browser sends `POST /transfer`. **BankServer** finds the session from the cookie, loads the user, checks the CSRF token, asks the **Router** for the matching route, and verifies that the user holds the permission the route requires (`SELF_SERVICE` for transfers). Only then does it call the handler in **AccountController**, which parses the form and calls **AccountService.transfer()**. The service runs the whole operation inside a unit of work, so it either completes fully or leaves nothing changed. Business errors travel back up as exceptions and are shown as a red note on the next page.

4. UML Diagrams

The diagrams below describe the system from several viewpoints: what users can do (use case), what the classes are and how they relate (class diagrams), how objects cooperate over time (sequence diagrams), how objects change state (state diagrams), and how data is stored (entity-relationship diagram). They were drawn from the actual source code, so the class names, methods and relationships match the project.

4.1 Use case diagram

There are three actors. A **Customer** manages their own money and loans. A **Teller** serves any customer at the counter. A **Manager** is a Teller (generalization arrow) who additionally approves loans, freezes accounts, credits interest, manages staff and reads the audit trail.

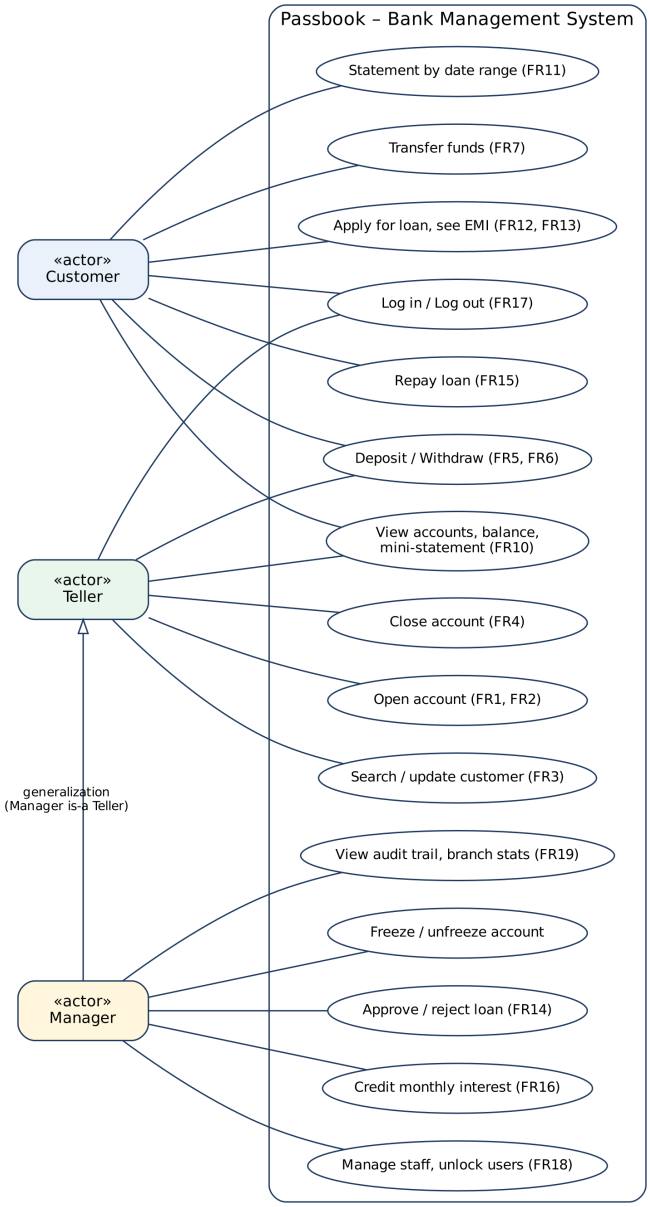


Figure 4.1: Use case diagram of Passbook

4.2 Class diagrams

The class model is shown in three parts so that each stays readable. Notation: a hollow triangle is inheritance (generalization), a dashed arrow with a hollow triangle is interface realization, a dashed open arrow is a dependency or use, and a solid line with multiplicities is an association.

4.2.1 User hierarchy

User is abstract: nobody is just a "user"; everyone is a Customer, Teller or Manager. **Manager extends Teller** because a manager can do everything a teller can. Each concrete class overrides `role()` and `permissions()`.

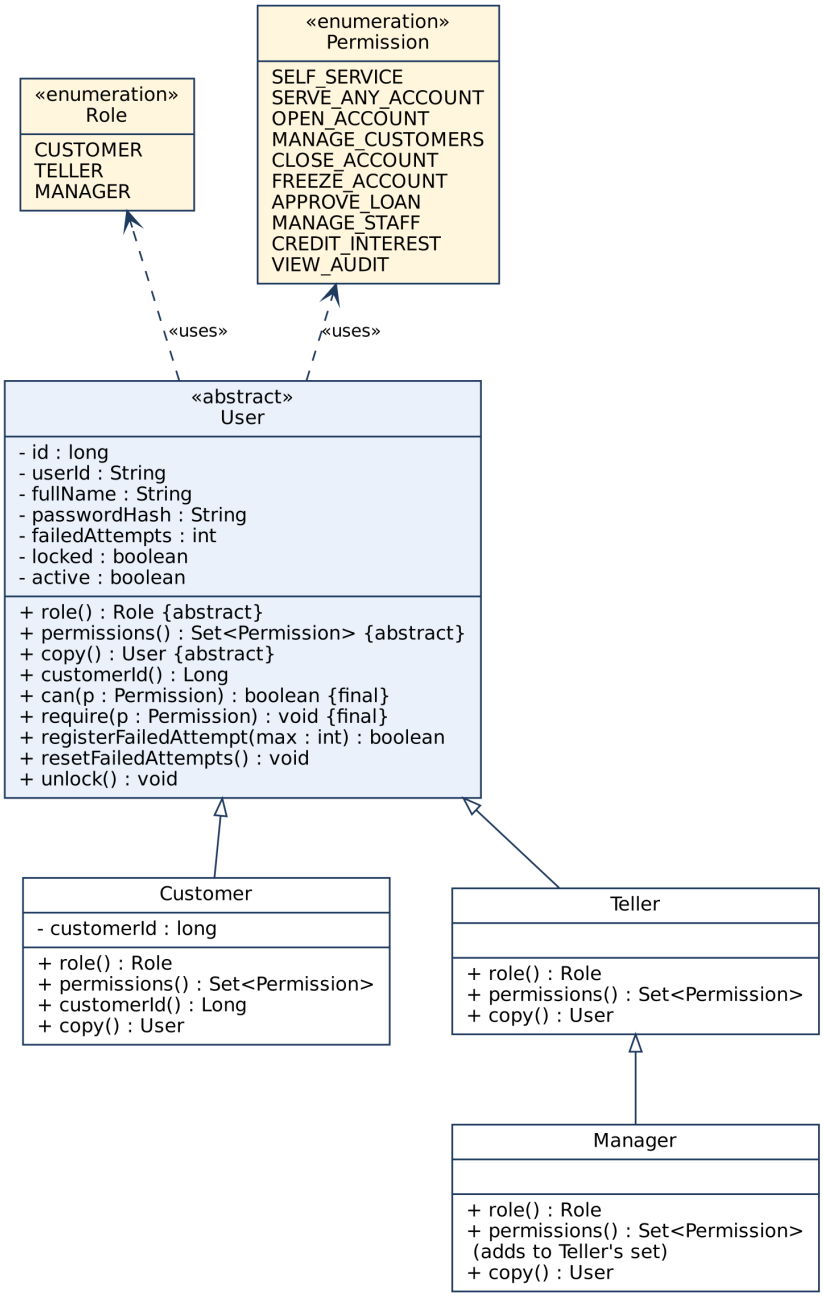


Figure 4.2: Class diagram, the User hierarchy

4.2.2 Account hierarchy

Account is abstract and holds everything that is common: the private balance, status, and the final methods `deposit()`, `withdraw()`, `close()`. **SavingsAccount** and **CurrentAccount** supply only what differs: the withdrawal rule. **InterestBearing** is an interface implemented only by savings accounts. **AccountFactory** creates the right subclass from a type name.

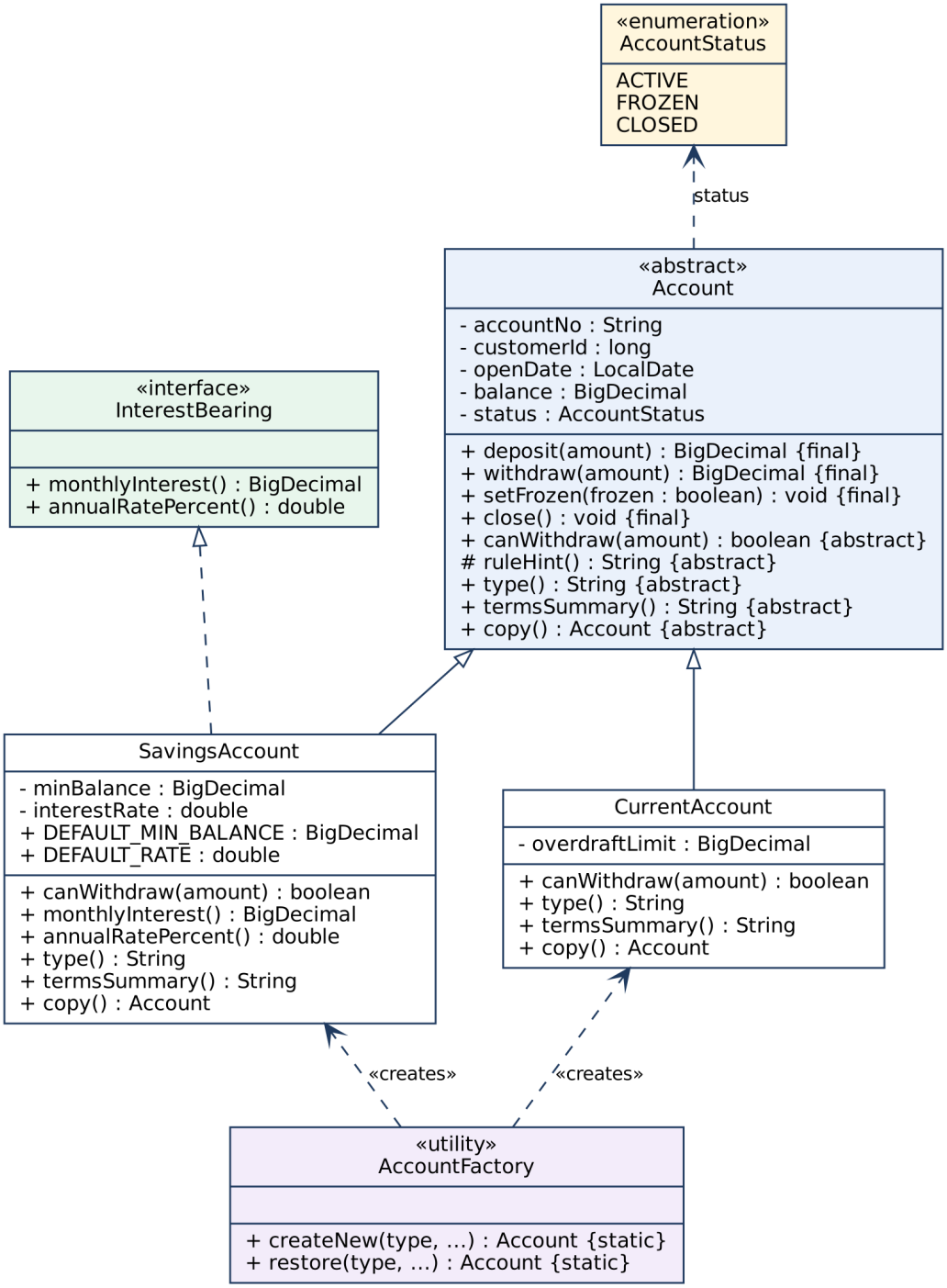


Figure 4.3: Class diagram, the Account hierarchy

4.2.3 Domain relationships

A customer profile owns many accounts; an account has a ledger of many transactions; a customer applies for many loans, each paid into and repaid from an account. **Transaction**, **CustomerProfile** and **AuditEntry** are Java records, which makes them immutable value objects.

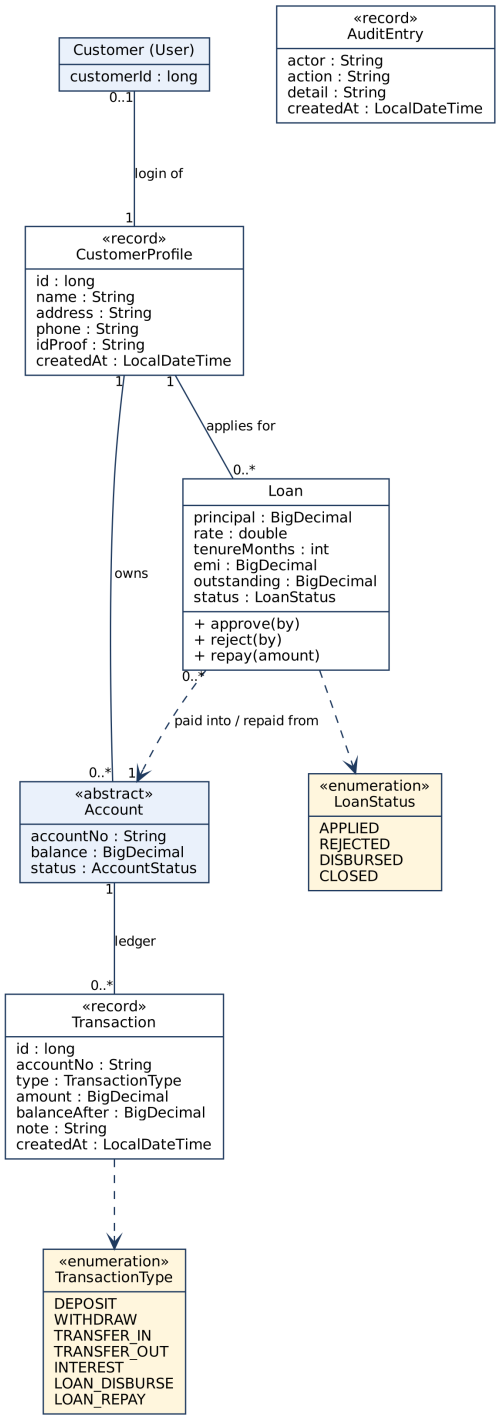


Figure 4.4: Class diagram, how the domain objects relate

4.2.4 Persistence and service classes (textual class diagram)

The service layer depends on interfaces, never on SQLite. This is what allows the same services to run against the SQLite repositories in production and the in-memory repositories in the tests.

```
<<interface>> UnitOfWork                  <<interface>> AccountRepository
  + call(Supplier<T>) : T                    + nextAccountNumber() : String
  + execute(Runnable) : void (default)       + insert(Account) / update(Account)
        ^                                    + findByNo(String) : Optional<Account>
        | implements                         + findByCustomer(long) : List<Account>
  SqliteUnitOfWork --> Database              + findAllOpen() : List<Account>
                                                    ^            ^
<<record>> Repositories                             |            |
  uow, users, customers, accounts,          SqliteAccountRepository   MemoryRepositories (tests)
  transactions, loans, audit

AccountService  --uses-->  Repositories, PasswordHasher, EventBus
LoanService     --uses-->  Repositories, EventBus
AuthService     --uses-->  Repositories, PasswordHasher, EventBus
AdminService    --uses-->  Repositories, PasswordHasher, EventBus

<<interface>> BankListener  <---implements---  AuditListener --> AuditRepository
EventBus  o--> BankListener (0..*)        (Observer: services publish, listeners react)

<<interface>> PasswordHasher  <---implements---  Pbkdf2PasswordHasher

<<abstract>> Controller  <---extends---  AuthController, DashboardController,
                                          AccountController, LoanController, AdminController
Router  o--> Route (method, pattern, permission, Handler)       <<functional>> Handler
```

4.3 Sequence diagrams

Sequence diagrams show objects as vertical lifelines and messages as horizontal arrows, read from top to bottom. Three important flows are shown: a transfer, a login, and a loan approval.

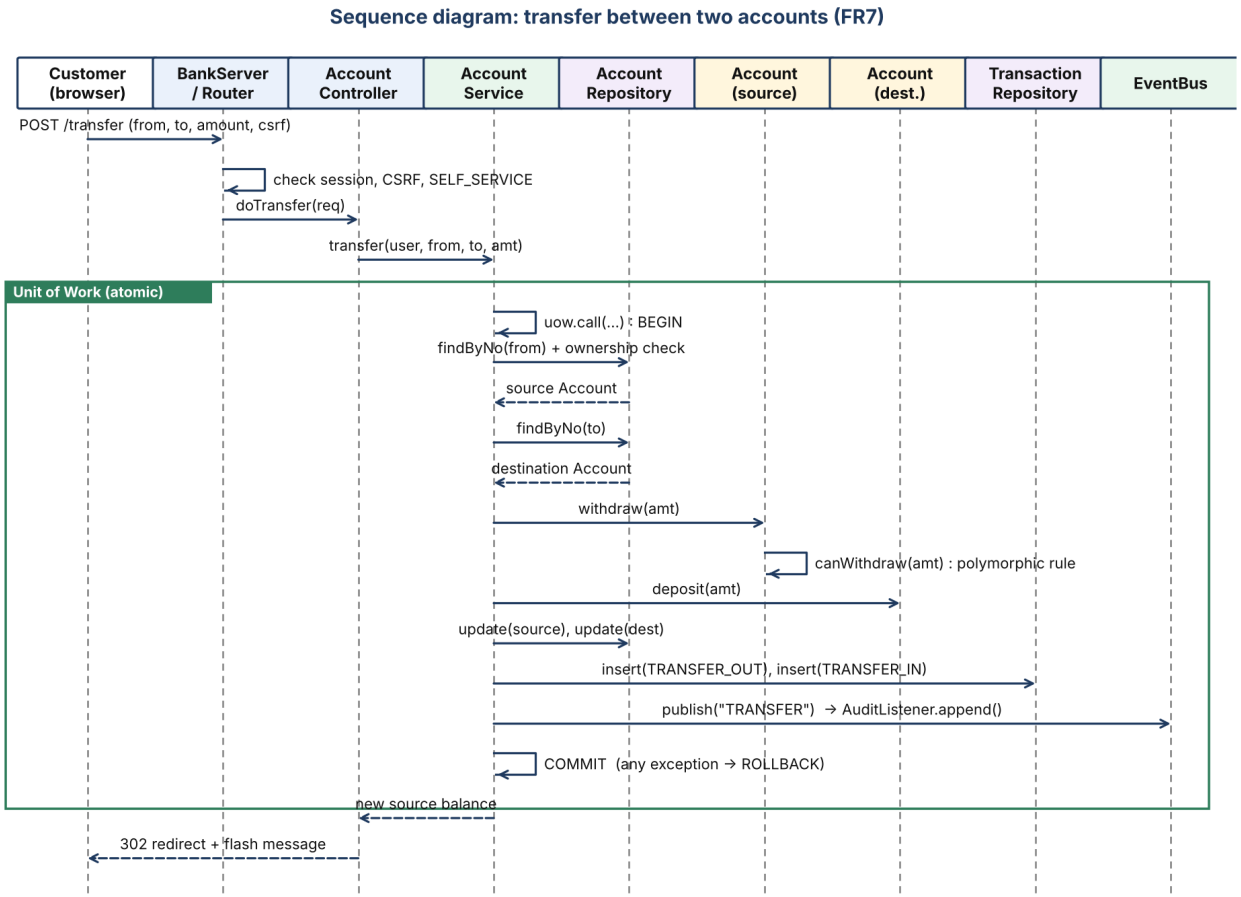


Figure 4.5: Sequence diagram, transfer between two accounts (FR7)

What to notice: the service opens a unit of work first; `withdraw()` on the source account triggers the polymorphic `canWithdraw()` check (a minimum-balance rule for savings, an overdraft rule for

current); if anything throws, nothing is saved. The event bus publishes the audit record inside the same transaction.

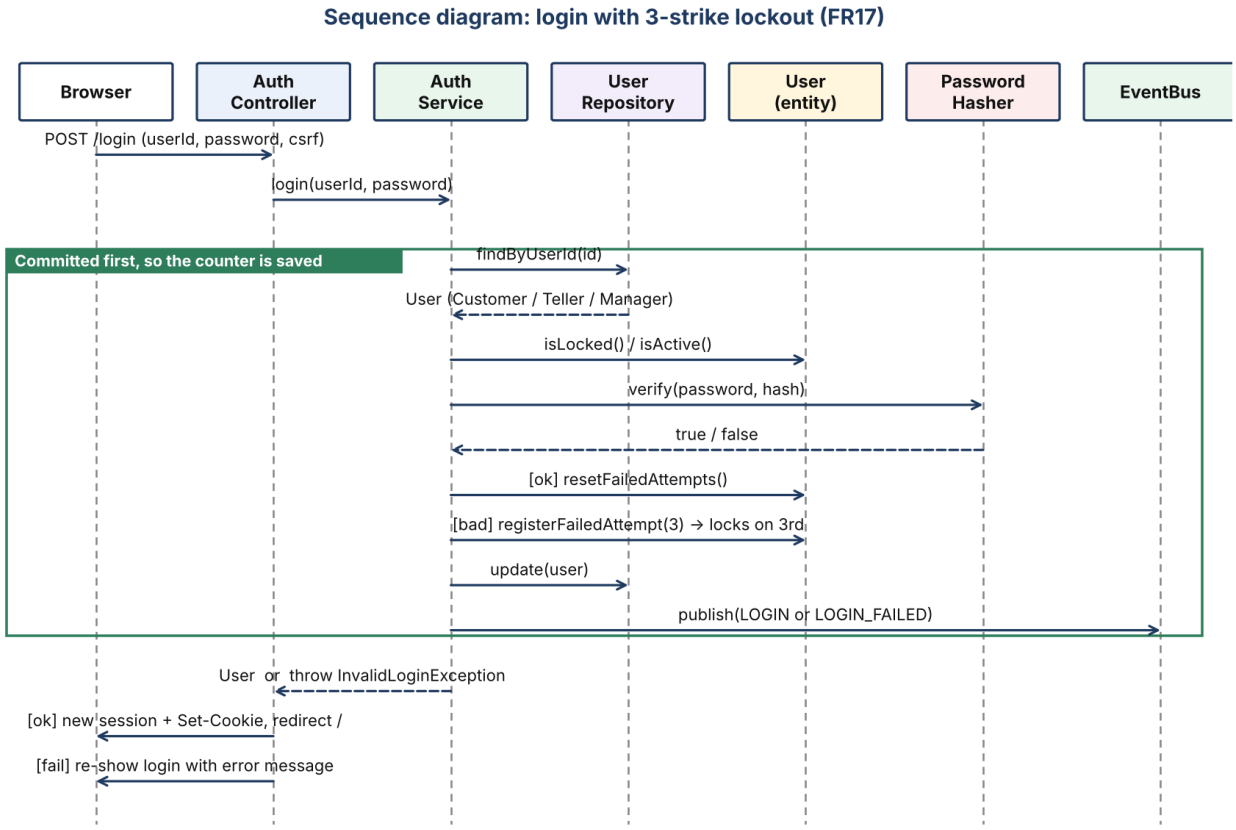


Figure 4.6: Sequence diagram, login with 3-strike lockout (FR17)

What to notice: the failed-attempt counter must survive the failed login, so `AuthService` commits the attempt first and throws `InvalidLoginException` only after the commit. The lock logic itself lives in the `User` entity.

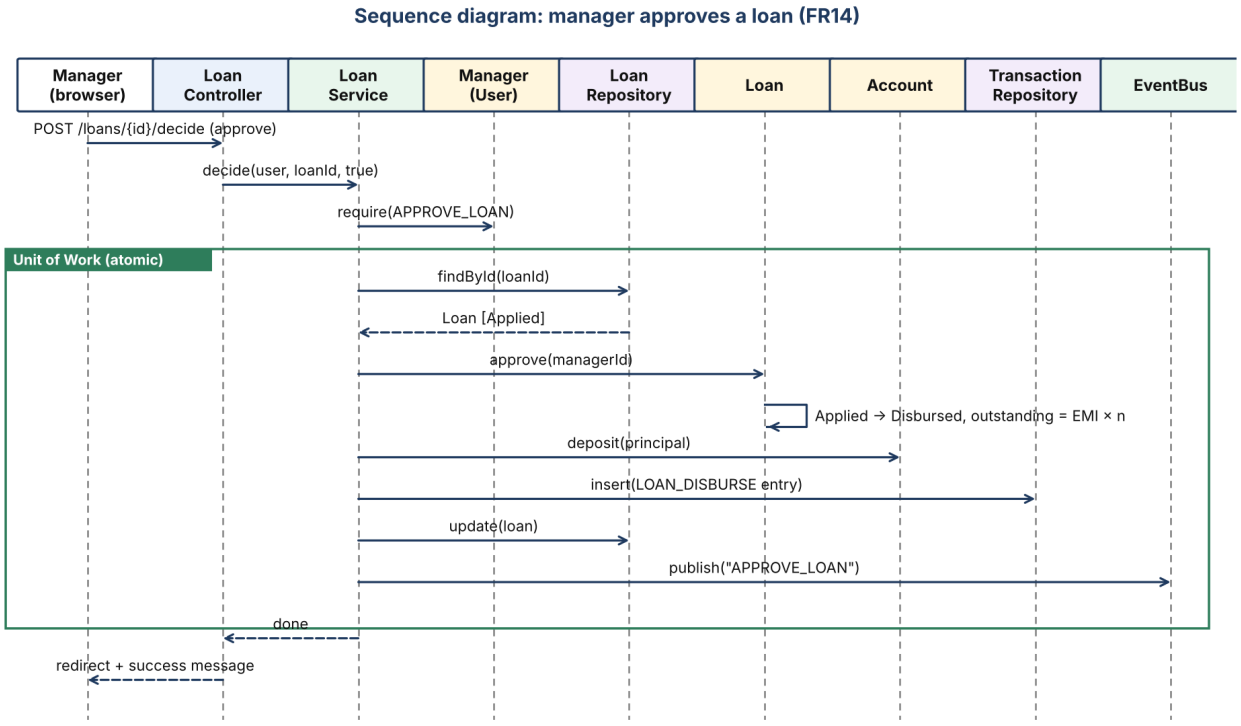


Figure 4.7: Sequence diagram, manager approves a loan (FR14)

4.4 State diagrams

Three objects have a life-cycle that is enforced by their own methods: Account, Loan, and User login state.

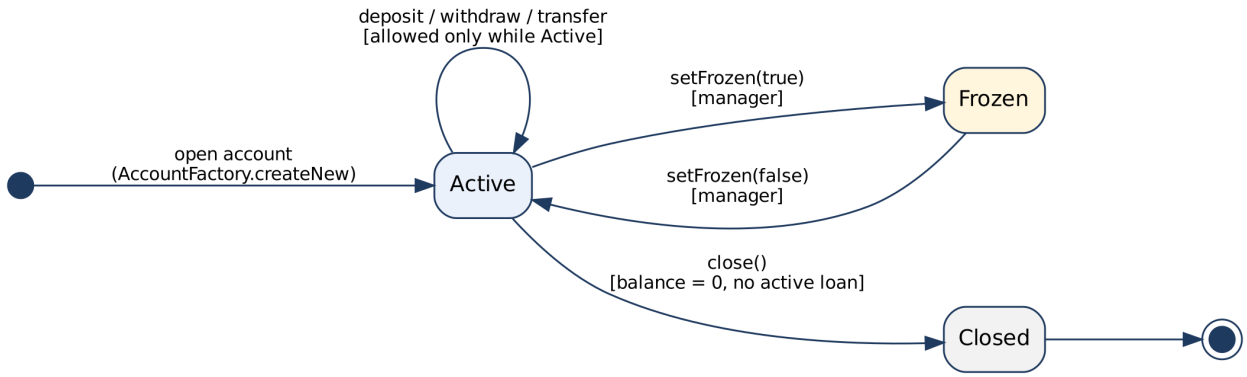


Figure 4.8: State diagram, Account status

An account can be used for money movement only while Active. A frozen account rejects deposits, withdrawals and transfers. Closing requires a zero balance and no active loan; a closed account can never change again.

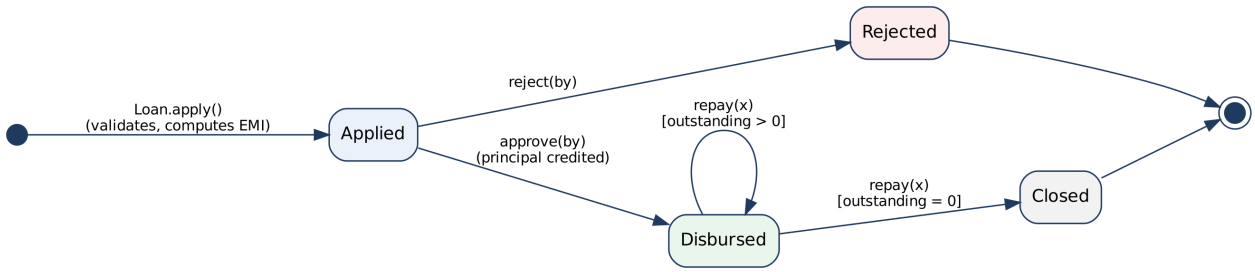


Figure 4.9: State diagram, Loan status

Approval disburses the principal in one step and sets the outstanding amount to EMI x tenure. Each repayment reduces it, and the loan closes when it reaches zero. Only an Applied loan can be approved or rejected.

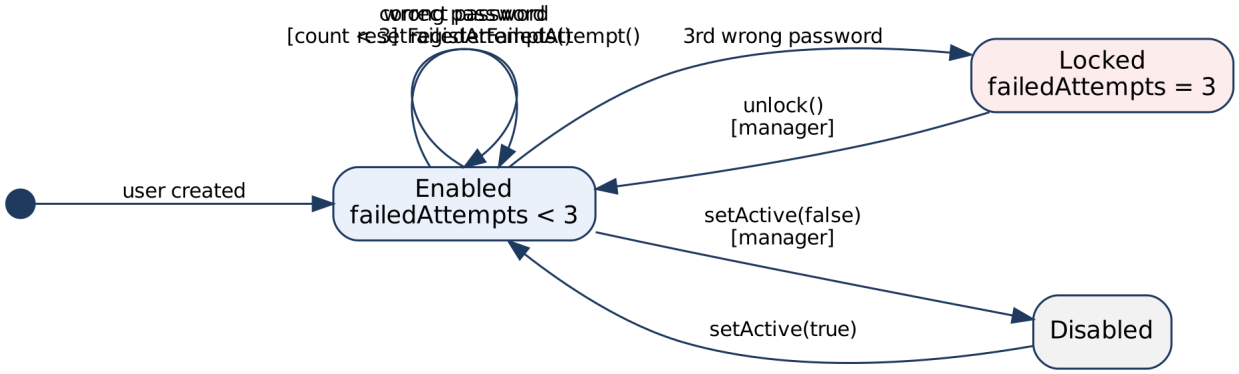


Figure 4.10: State diagram, User login state

4.5 Database design (ER diagram)

The database has six tables. Foreign keys and CHECK constraints enforce the same rules as the Java code, as a second line of defence (for example, `type IN ('savings','current')` and the loan and account status values).

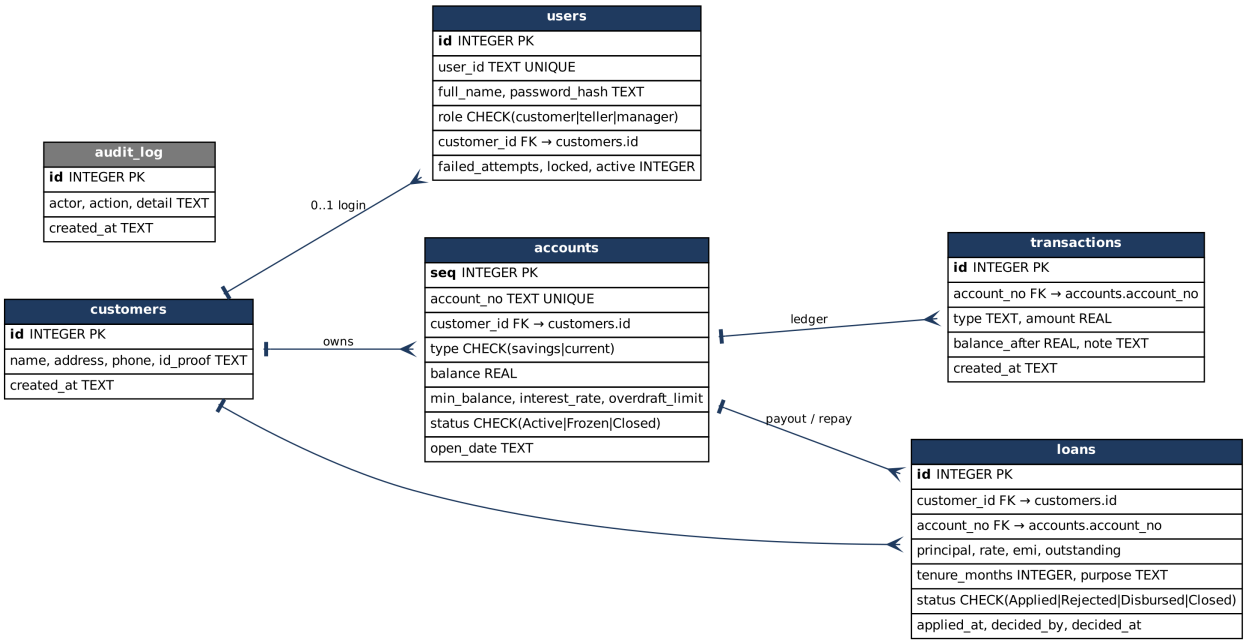


Figure 4.11: Entity-relationship diagram of bank.db

Design note: the class hierarchy Savings/Current maps to a single `accounts` table with a `type` column (single-table inheritance). Columns that apply to only one subclass (`min_balance`, `interest_rate`, `overdraft_limit`) are nullable. `AccountFactory.restore()` rebuilds the right subclass when rows are read.

4.6 Component diagram

A component diagram shows the large replaceable building blocks of the system and how they depend on each other. In Passbook each Java package is one component. A dashed arrow marked «use» means one component calls the other; the dashed arrow with a hollow head means a component realizes (implements) the interfaces of another.

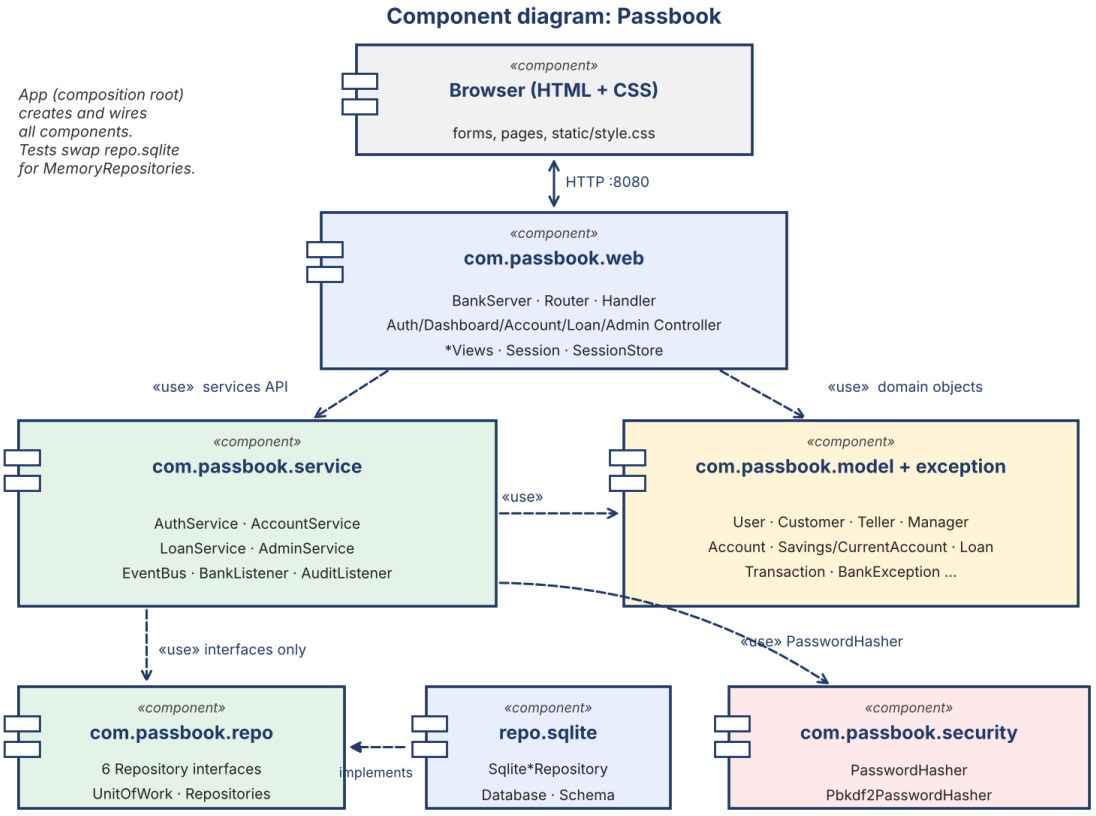


Figure 4.12: Component diagram of Passbook

What to notice: the web component talks only to the service component and the domain objects, never to the database. The service component depends on *repo* (interfaces only), so *repo.sqlite* can be replaced by the in-memory repositories used in the tests without changing any service. *security* is a separate component, which is why the password algorithm can be changed in one place. *App* is the composition root that creates and connects every component.

4.7 Collaboration diagram

A collaboration (communication) diagram shows the same interaction as a sequence diagram, but the objects are laid out freely and numbered messages show the order. It emphasizes which objects are linked to which. The diagram below follows the transfer use case (FR7), the same flow as the sequence diagram in Figure 4.5.

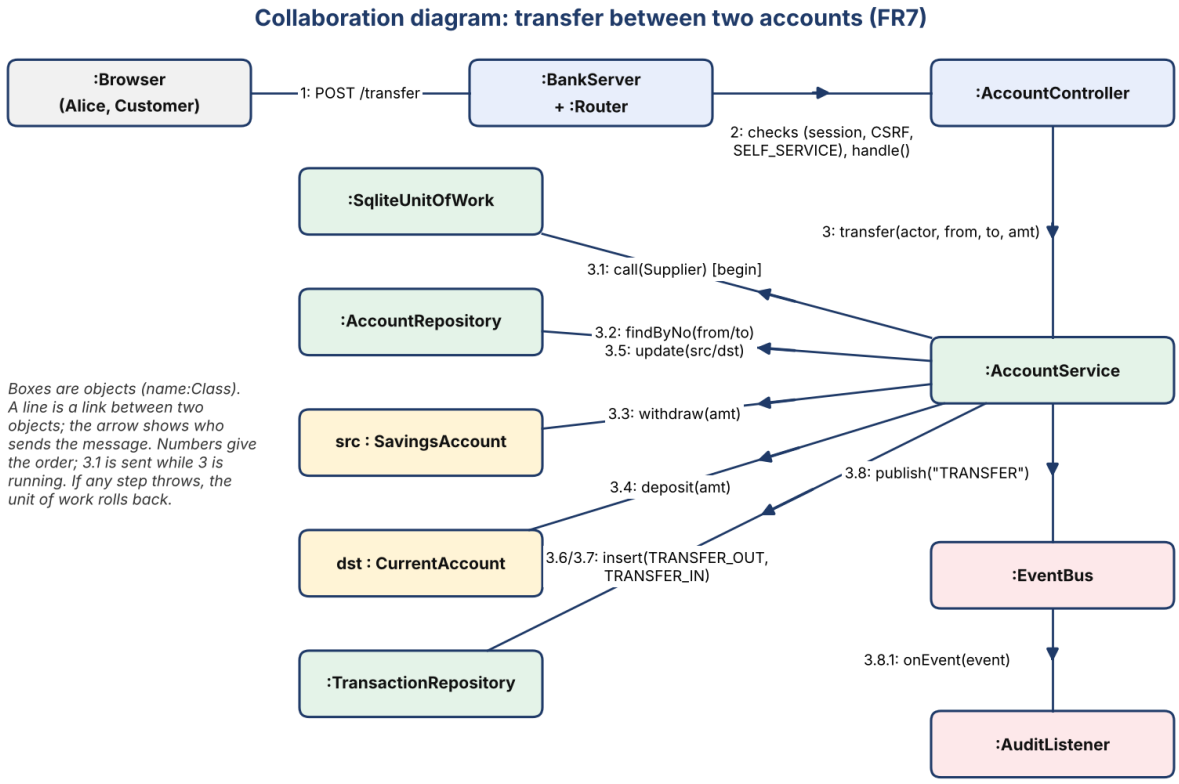


Figure 4.13: Collaboration diagram, transfer between two accounts (FR7)

What to notice: *AccountService* is the hub of the collaboration. It is linked to the unit of work, both accounts, the repositories and the event bus, which is why the service layer holds the business flow while the controller only forwards the request. Message 3.3 is polymorphic: *src* is declared as Account, and its own *canWithdraw()* rule runs. Message 3.8.1 is the Observer pattern: the service knows only the event bus, not the audit listener. Numbers 3.1 to 3.8.1 are nested inside message 3, so if one of them throws, the unit of work rolls everything back.

4.8 Object diagram

A class diagram describes all possible objects; an object diagram shows one real snapshot of the running system, with actual values. The snapshot below uses the demo data (Alice Verma with a savings and a current account, opened by the manager) after Alice transfers 2,000.00 from her savings account to her current account.

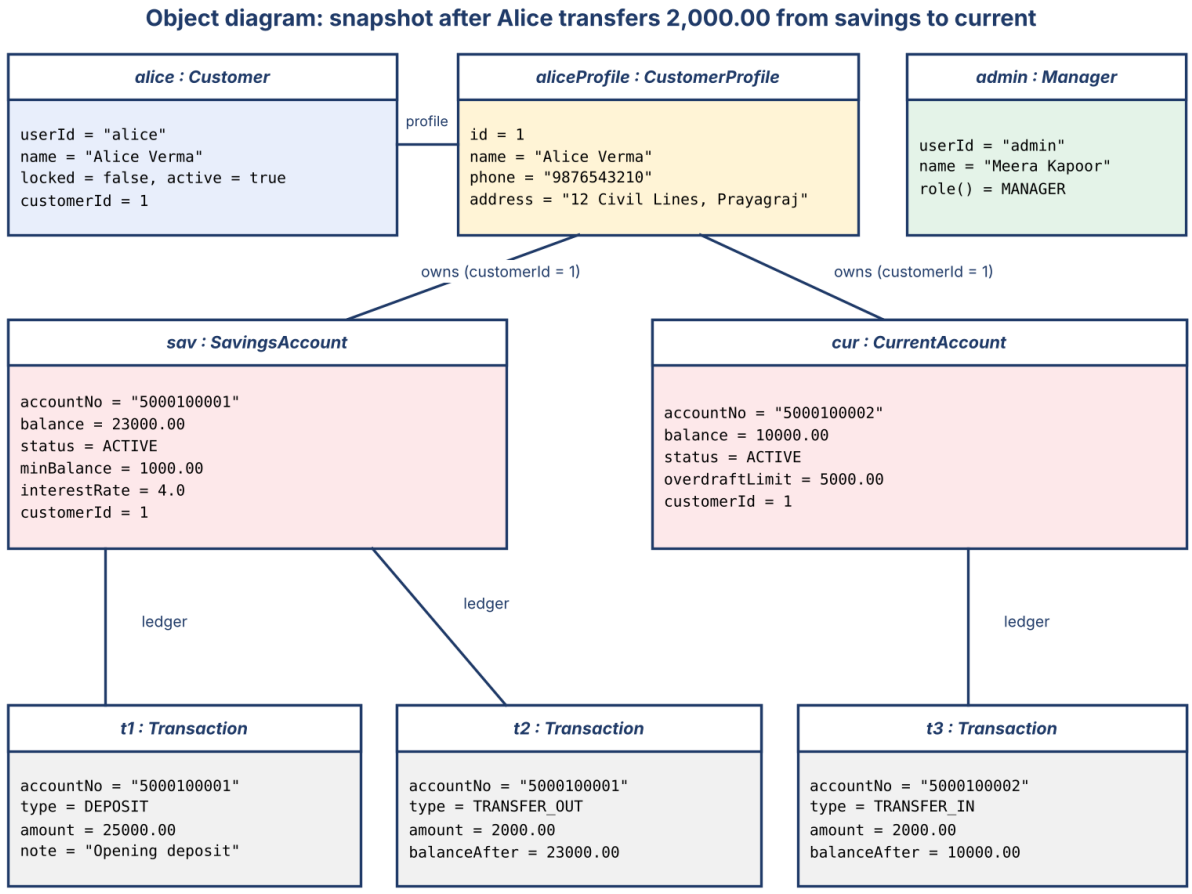


Figure 4.14: Object diagram of Passbook after one transfer

What to notice: the transfer left exactly two new ledger objects, *t2* (TRANSFER_OUT on savings) and *t3* (TRANSFER_IN on current), and the two balances moved by the same amount: 25,000.00 - 2,000.00 = 23,000.00 and 8,000.00 + 2,000.00 = 10,000.00. The savings object keeps its minimum balance of 1,000.00 and the current object its overdraft limit of 5,000.00, so each object carries the data that its own subclass rule uses. Both accounts point to the same customer profile (customerId = 1).

5. Object-Oriented Concepts and Where They Are Used

This section explains each concept in simple words, shows the exact place in the Passbook code where it is used, and says why the design benefits from it. Class and method names are real and can be found in the source files named in each part.

Concept	One-line meaning	Main place in Passbook
Abstraction	Expose what something does, hide how	Account, User (abstract); InterestBearing, PasswordHasher, *Repository, UnitOfWork (interfaces)
Encapsulation	Keep data private; change it only via methods that enforce rules	Account.balance; User lockout fields; Loan state
Inheritance	A subclass reuses and extends a parent	Savings/Current extend Account; Customer/Teller extend User; Manager extends Teller
Polymorphism	Same call, different behaviour by actual type	canWithdraw(), permissions(), termsSummary(), copy(), Controller.register()
Abstract class	Partly built class that cannot be instantiated	Account, User, Controller
Interface	A contract with no implementation	InterestBearing, BankListener, PasswordHasher, Handler, 6 repositories, UnitOfWork

5.1 Abstraction

Meaning. Abstraction means showing only what is needed and hiding the details. The rest of the program works with a simple idea, and the complicated or changeable part stays behind it.

Where it is used.

- **Account** is the abstraction of "a bank account". The services never care whether it is savings or current; they call `withdraw()` and the details stay hidden.
- **User** is the abstraction of "someone who can log in". Code asks `user.can(Permission.APPROVE_LOAN)` and does not check the class.
- **PasswordHasher** hides the hashing algorithm. `AuthService` only calls `hash()` and `verify()`; the PBKDF2 details are inside one class.
- **Repository interfaces** hide SQL. `AccountService` calls `accounts.findByNo(no)` and has no idea whether the data comes from SQLite or memory.
- **UnitOfWork** hides transaction handling behind one method, `call(Supplier)`.

```
// model/User.java  -- abstract: subclasses decide role and permissions
public abstract class User {
    public abstract Role role();
    public abstract Set<Permission> permissions();
    public final boolean can(Permission p) { return permissions().contains(p); }
}
```

Benefit. New account types, a stronger hash algorithm, or a different database can be added without editing the services.

5.2 Encapsulation

Meaning. Encapsulation means keeping an object's data private and allowing changes only through methods that protect the rules. The object guards its own correctness.

Where it is used.

- **Account.balance** is `private`. There is no `setBalance()`. The balance changes only through `deposit()` and `withdraw()`, which validate the amount, check the account is active, and apply the withdrawal rule.
- **Account.status** changes only through `setFrozen()` and `close()`. Closing refuses unless the balance is zero.
- **User lockout state** (`failedAttempts`, `locked`) changes only through `registerFailedAttempt()`, `resetFailedAttempts()` and `unlock()`. The password hash has a getter but no setter.
- **Loan** owns its life-cycle. Status changes only through `approve()`, `reject()` and `repay()`; each refuses an illegal transition. IDs can be assigned once only (`assignId()` throws if already set).
- **CustomerProfile** (a record) validates in its constructor, so an invalid profile can never exist.
- **Money** keeps all money rules (BigDecimal, 2 decimals, maximum amount) in one private-constructor utility class.

```
// model/Account.java
private BigDecimal balance;                 // hidden from the outside

public final BigDecimal withdraw(BigDecimal amount) {
    BigDecimal a = Money.validatedAmount(amount);   // rule 1: positive, within limit
    requireActive();                                 // rule 2: not frozen or closed
    if (!canWithdraw(a)) {                           // rule 3: subtype-specific rule
        throw new InsufficientFundsException(accountNo, a, ruleHint());
    }
    balance = balance.subtract(a);
    return a;
}
```

Benefit. It is impossible for any other class, including the web layer, to put an account into an invalid state. The rules live in one place, so a bug cannot be introduced by forgetting a check somewhere else.

5.3 Inheritance

Meaning. Inheritance lets a class (subclass) reuse the fields and methods of another class (superclass) and add or change behaviour. It models an "is-a" relationship.

Where it is used.

Parent	Children	What is shared / what is added
Account (abstract)	SavingsAccount, CurrentAccount	Shared: balance, status, deposit(), withdraw(), close(). Added: min balance and interest (savings); overdraft limit (current)
User (abstract)	Customer, Teller	Shared: id, userId, name, password hash, lockout logic. Added: customerId (Customer); staff permissions (Teller)
Teller	Manager	A manager is a teller plus extra permissions (multi-level inheritance: Manager > Teller > User)
BankException	InsufficientFundsException, AccountNotFoundException, InvalidLoginException, AccessDeniedException	One root type lets the web layer catch every business error with one catch block
Controller (abstract)	AuthController, DashboardController, AccountController, LoanController, AdminController	Shared helpers: ctx(), flash(), attempt(), safeLocal(). Each adds its own routes

```
// model/Manager.java  -- three levels: User > Teller > Manager
public class Manager extends Teller {
    @Override public Role role() { return Role.MANAGER; }

    @Override
    public Set<Permission> permissions() {
        Set<Permission> p = EnumSet.copyOf(super.permissions());   // reuse the Teller's set
        p.addAll(EnumSet.of(Permission.FREEZE_ACCOUNT, Permission.APPROVE_LOAN,
                 Permission.MANAGE_STAFF, Permission.CREDIT_INTEREST, Permission.VIEW_AUDIT));
        return p;
    }
}
```

Benefit. Common code is written once. When the manager's powers change, only `Manager.permissions()` is edited, and the teller's powers are inherited automatically.

5.4 Polymorphism

Meaning. Polymorphism means one call can behave differently depending on the real object behind it. The caller uses the parent type; Java picks the child's version at run time (method overriding).

Where it is used.

Polymorphic call	Behaviour per type
Account.canWithdraw(amount)	**Savings**: balance minus amount must stay at or above the minimum balance. **Current**: balance plus overdraft limit must cover the amount
Account.termsSummary()	Savings: "4.0% p.a. - min 1,000.00". Current: "Overdraft up to 5,000.00". Shown on the passbook cover
Account.type() and ruleHint()	Return "savings" / "current" and the error hint text that fits the type
User.permissions()	Customer: self-service only. Teller: serve any account, open accounts, manage customers, close accounts. Manager: all of the teller's plus freeze, approve loans, staff, interest, audit
User.role()	Returns CUSTOMER, TELLER or MANAGER
Account.copy() / User.copy()	Each subclass returns a copy of its own type (Prototype pattern, used by the in-memory test store)
Controller.register(Router)	App loops over a `List<Controller>` and calls register(); each controller adds its own routes
BankListener.onEvent(event)	EventBus calls every listener the same way; AuditListener writes to the audit log (more listeners can be added)

```
// service/AccountService.java  -- one line works for every account type
BigDecimal amt = src.withdraw(amount);   // src is declared as Account
dst.deposit(amt);

// service/AccountService.java  -- monthly interest: only some accounts qualify
for (Account a : accounts.findAllOpen()) {
    if (a instanceof InterestBearing && a.getStatus() == AccountStatus.ACTIVE) {
        BigDecimal interest = ((InterestBearing) a).monthlyInterest();
        ...
    }
}

// App.java  -- polymorphism over controllers
for (Controller c : controllers) c.register(router);
```

Benefit. The transfer code is identical for any two accounts, yet a savings account refuses to go below its minimum while a current account may overdraw. Adding a third account type (for example a fixed deposit) means writing one new subclass and registering it in `AccountFactory`; the services do not change. The test "*polymorphism: each account type applies its own withdrawal*

rule" checks exactly this.

Compile-time polymorphism (overloading). `Response.html(String)` and `Response.html(int, String)` are overloads, as are the two `Router.route(...)` methods (with and without a required permission).

5.5 Abstract Classes

Meaning. An abstract class is an incomplete parent. It contains finished code that all children share and also declares abstract methods that every child must complete. It cannot be instantiated with new.

Abstract class	Concrete (finished) members	Abstract members children must supply
Account	deposit(), withdraw(), setFrozen(), close(), getters, requireActive()	canWithdraw(), ruleHint(), type(), termsSummary(), copy()
User	can(), require(), registerFailedAttempt(), resetFailedAttempts(), unlock(), getters	role(), permissions(), copy()
Controller	ctx(), flash(), attempt(), intField(), safeLocal()	register(Router)

Template Method pattern. `Account.withdraw()` is a template method. It is `final` so no subclass can change the sequence (validate amount, check active, check rule, subtract), but one step, `canWithdraw()`, is abstract and filled in by the subclass. Marking `deposit()`, `withdraw()`, `close()` and `setFrozen()` as `final` stops a subclass from weakening the rules.

Why abstract and not concrete? A plain "Account" or plain "User" has no meaning in a bank: its withdrawal rule or its permissions are undefined. Making the class abstract forbids creating such a meaningless object, and the compiler forces every new subclass to answer the open questions.

5.6 Interfaces

Meaning. An interface is a contract: a list of methods a class promises to provide, with no code of its own (apart from optional default methods). A class can implement many interfaces, which is how Java gets multiple inheritance of type.

Interface	Implemented by	Purpose
InterestBearing	SavingsAccount only	Capability: "earns interest" (monthlyInterest(), annualRatePercent()). Current accounts do not have it, so they are skipped when interest is credited
BankListener	AuditListener	Observer pattern: anything that wants to hear about bank events
PasswordHasher	Pbkdf2PasswordHasher	Swap the hashing algorithm without touching AuthService or AccountService
UserRepository, CustomerRepository, AccountRepository, TransactionRepository, LoanRepository, AuditRepository	Sqlite*Repository (production), MemoryRepositories (tests)	Hide storage; services depend on the interface only
UnitOfWork	SqliteUnitOfWork, in-memory version	Atomic commit / rollback; has a default method execute(Runnable) built on call()
Handler (functional)	Lambdas / method references such as this::doTransfer	One-method interface so route handlers can be written as lambdas

```
// model/InterestBearing.java
public interface InterestBearing {
    BigDecimal monthlyInterest();
    double annualRatePercent();
}
// model/SavingsAccount.java
public class SavingsAccount extends Account implements InterestBearing { ... }
```

Abstract class versus interface in this project. Account is an abstract class because savings and current accounts share real state and behaviour (balance, status, deposit, withdraw). **InterestBearing** is an interface because earning interest is a capability that cuts across the hierarchy, not a kind of account. Repositories are interfaces because they carry no state and several unrelated implementations exist.

Interfaces also make the system testable. Because services use the repository interfaces, the 31 tests run with an in-memory implementation and need no database at all.

5.7 Other OOP and Java Features Used

Feature	Where in Passbook
Method overriding	role(), permissions(), canWithdraw(), termsSummary(), copy(); Manager.permissions() extends Teller.permissions() via super
Method overloading	Response.html(String) / html(int, String); Router.route(method, pattern, handler) / route(method, pattern, permission, handler)
Constructors and super()	SavingsAccount, CurrentAccount call super(...) to initialise the shared Account fields; User has a protected constructor so only subclasses can create users
Access modifiers	private fields, protected abstract ruleHint() (visible to subclasses only), protected User constructor, package-private SQLite classes (Database, Schema)
static and final	static utility Money and factories (private constructors); static Loan.apply() and Loan.calculateEmi(); final template methods; final fields for immutable data
Enums with behaviour	Role, Permission, AccountStatus (label), LoanStatus (label), TransactionType (isCredit(), label())
Records	Transaction, CustomerProfile (with validation), AuditEntry, BankEvent, Repositories, OpenedAccount: immutable data carriers
Exceptions	BankException hierarchy; unit of work rolls back on any exception; web layer turns BankException into a red message, AccessDeniedException into a 403 page
Composition / aggregation	AccountService is composed of repositories, the hasher and the event bus; EventBus holds a list of listeners; Router holds routes
Generics and lambdas	UnitOfWork.call(Supplier<T>), Database.query(RowMapper<T>), Optional<Account>, List<Controller>; Handler lambdas and method references
Dependency injection	Every service receives its dependencies through the constructor; App wires everything by hand

6. Exception Handling

All business-rule violations extend one root class, **BankException** (a RuntimeException). Programming bugs are different and are not BankExceptions, so the server can tell the user "you entered something wrong" apart from "something broke".

Exception	Thrown when	What the user sees
BankException	Any rule violation: bad amount, frozen account, closing with balance, invalid loan terms	Red message on the page (HTTP 400 if uncaught)

Exception	Thrown when	What the user sees
InsufficientFundsException	Withdraw or transfer would break the account's rule (minimum balance or overdraft)	Message that includes the account's rule hint
AccountNotFoundException	Account number does not exist	404 page "No such account"
InvalidLoginException	Wrong user ID or password, locked or disabled user	Error on the login page
AccessDeniedException	A role tries something it has no permission for, or touches another customer's account	403 page "Not for your role"

Atomicity. When an exception is thrown inside `uow.call(...)`, the database transaction is rolled back. Test *TC7b* proves it: a transfer into a frozen destination leaves **both** balances unchanged.

7. Design Patterns Used

Pattern	Where	Problem it solves
Factory	AccountFactory, UserFactory	Callers ask for "savings" or a role by name and never call constructors; rebuilding objects from database rows uses the same code
Template Method	Account.withdraw() with abstract canWithdraw()	Fixed sequence of steps with one step that varies by subclass
Observer	EventBus, BankListener, AuditListener	Services publish events; the audit log reacts without services knowing about it
Repository	*Repository interfaces + Sqlite* implementations	Business code never contains SQL
Unit of Work	UnitOfWork, SqliteUnitOfWork	Several changes commit or roll back as one
Prototype	Account.copy(), User.copy(), Loan.copy()	Independent copies for the in-memory store used by tests
Dependency Injection	Constructors of all services; App	Parts are connected from outside, so they can be replaced in tests
MVC	web/*Controller (C), web/*Views (V), service + model (M)	Separates request handling, page generation and business rules
Front controller	BankServer + Router	One entry point performs session, CSRF and permission checks for every route

8. Security and Reliability Features

- **Password storage:** PBKDF2-HMAC-SHA256 with a random 16-byte salt and 65,536 iterations, stored as `pbkdf2$iterations$salt$hash`. Verification uses a constant-time comparison.
- **Login lockout:** three wrong passwords lock the user; only a manager can unlock (FR17, FR18).
- **Sessions:** random ids, 10-minute idle timeout, a fresh session id after login (prevents session fixation), and sessions destroyed if the user is disabled or locked meanwhile.
- **CSRF protection:** every POST must carry the session's token, compared in constant time.
- **XSS protection:** all user text goes through `Html.esc()` before it is placed in a page.
- **Authorization:** each route declares the permission it needs; services check again with `actor.require(...)`; account access is limited to the owner or staff.
- **Open-redirect and path-traversal guards:** only same-site relative redirect targets are accepted; static file paths are normalised and must stay inside the static folder.

- **Money correctness:** all amounts are BigDecimal with 2 decimals (never double), with a maximum single-transaction limit.
- **Consistency under concurrency:** one database connection guarded by a re-entrant lock serialises requests, so two simultaneous transfers cannot lose or create money.
- **Audit trail:** every important action (login, deposit, transfer, approval, unlock) is written to the audit log by the Observer.

9. Testing

The project contains 31 tests in `BankTests` (business logic, 20 tests) and `WebTests` (HTTP layer, 11 tests). They run with `./test.sh` using the in-memory repositories, so no database is needed, and the web tests start the real server on a free port. The script `scripts/check_sql.py` additionally validates every SQL statement against a real SQLite engine.

Test	What it proves
TC1, TC2	Deposit adds to the balance and writes a ledger entry; withdrawal within balance works
TC3, TC4	Savings minimum balance blocks a withdrawal; current account may overdraw up to its limit and not beyond (polymorphism)
TC5, TC6, TC7, TC7b	Transfer writes two ledger entries; insufficient funds changes nothing; unknown account fails cleanly; failure on the second step rolls back both sides
TC8	Closing needs a zero balance; closed accounts reject transactions
TC9	Three wrong passwords lock the user; a manager unlocks
Loans and interest	Loan life-cycle (apply, approve, repay, close); rejection leaves the balance alone; monthly interest goes to savings only
Roles and audit	Polymorphic permissions and access control; audit trail records critical actions
Concurrency	Concurrent transfers never lose or create money
Security (web)	Anonymous visitors redirected to login; CSRF refused; open redirect and path traversal blocked; HTML in input is escaped; customers cannot reach staff pages or other people's accounts; logout ends the session
Role flows (web)	Customer dashboard, transfer, statement; teller opens accounts and cannot do manager actions; manager loan desk, staff admin, audit, interest; lockout over HTTP

10. Limitations and Future Scope

- Loan approval pays out in one step, the loan rate is fixed at 10% p.a., and interest is credited by a manager button rather than automatically on a schedule.
- The "Dormant" state of the state diagram is not automated.
- The application models a single branch and a single currency.
- Balances are stored as REAL in SQLite, while Java uses BigDecimal; a future version could store money as integer minor units (paise) to remove any rounding risk at the database level.
- For real deployment the server should run behind HTTPS.
- Possible extensions: fixed-deposit and recurring-deposit account types (one new subclass each), an automatic interest scheduler, e-mail or SMS notifications as new BankListener implementations, and standing instructions.

Because of the interface-based design, most of these extensions need new classes rather than changes to existing ones, which is the main benefit of the object-oriented structure.

11. Conclusion

Passbook shows that object-oriented design is not only vocabulary but a practical way to organise a banking system. **Abstraction** (Account, User, repositories) hides what can change. **Encapsulation** keeps balances, loans and login state valid by letting objects guard their own data. **Inheritance** (Account and User hierarchies, Manager extends Teller) removes duplicate code. **Polymorphism** lets one transfer routine serve every account type and one permission check serve every role. **Abstract classes** fix the shared algorithm and leave the variable step to subclasses, and **interfaces** decouple the business logic from storage, hashing and auditing, which also made the system easy to test. Together with the UML models in this report, they give a design that is correct, secure, and ready to grow.

Appendix A. Source File Inventory

Package	Files
model (24)	User, Customer, Teller, Manager, Role, Permission, UserFactory, Account, SavingsAccount, CurrentAccount, InterestBearing, AccountStatus, AccountFactory, Transaction, TransactionType, Loan, LoanStatus, CustomerProfile, AuditEntry
exception (5)	BankException, InsufficientFundsException, AccountNotFoundException, InvalidLoginException, AccessDeniedException
repo (10) + repo.sqlite (10)	UserRepository, CustomerRepository, AccountRepository, TransactionRepository, LoanRepository, AuditRepository, UnitOfWork, Repositories; Database, Schema, SqliteUnitOfWork, SqliteRepositories and one Sqlite*Repository per interface
service (14)	AuthService, AccountService, LoanService, AdminService, EventBus, BankListener, BankEvent, AuditListener, DemoData, and small data carriers (OpenAccountRequest, OpenedAccount, AccountDetails, CustomerDetails, LoanView, Stats)
security (2)	PasswordHasher, Pbkdf2PasswordHasher
web (19)	BankServer, Router, Handler, Request, Response, Session, SessionStore, Flash, ViewContext, Controller, AuthController, DashboardController, AccountController, LoanController, AdminController, Html, Layout, AuthViews, BankingViews, LoanViews, AdminViews
util, root, tests	Money; App; tests: BankTests, WebTests, AllTests, T, MemoryRepositories, DevServer
other	pom.xml, run.sh / run.bat, test.sh / test.bat, static/style.css, scripts/check_sql.py, README.md